

EC200A-CN(TA) QuecOpen A/B 系统升级指导

LTE Standard 模块系列

版本：1.0.0

日期：2022-12-06

状态：临时文件



上海移远通信技术股份有限公司（以下简称“移远通信”）始终以为客户提供最及时、最全面的服务为宗旨。如需任何帮助，请随时联系我司上海总部，联系方式如下：

上海移远通信技术股份有限公司
上海市闵行区田林路 1016 号科技绿洲 3 期（B 区）5 号楼 邮编：200233
电话：+86 21 5108 6236 邮箱：info@quectel.com

或联系我司当地办事处，详情请登录：<http://www.quectel.com/cn/support/sales.htm>。

如需技术支持或反馈我司技术文档中的问题，请随时登陆网址：
<http://www.quectel.com/cn/support/technical.htm> 或发送邮件至：support@quectel.com。

前言

移远通信提供该文档内容以支持客户的产品设计。客户须按照文档中提供的规范、参数来设计产品。同时，您理解并同意，移远通信提供的参考设计仅作为示例。您同意在设计您目标产品时使用您独立的分析、评估和判断。在使用本文档所指导的任何硬软件或服务之前，请仔细阅读本声明。您在此承认并同意，尽管移远通信采取了商业范围内的合理努力来提供尽可能好的体验，但本文档和其所涉及服务是在“可用”基础上提供给您的。移远通信可在未事先通知的情况下，自行决定随时增加、修改或重述本文档。

使用和披露限制

许可协议

除非移远通信特别授权，否则我司所提供硬软件、材料和文档的接收方须对接收的内容保密，不得将其用于除本项目的实施与开展以外的任何其他目的。

版权声明

移远通信产品和本协议项下的第三方产品可能包含受移远通信或第三方材料、硬软件和文档版权保护的相关资料。除非事先得到书面同意，否则您不得获取、使用、向第三方披露我司所提供的文档和信息，或对此类受版权保护的资料进行复制、转载、抄袭、出版、展示、翻译、分发、合并、修改，或创造其衍生作品。移远通信或第三方对受版权保护的资料拥有专有权，不授予或转让任何专利、版权、商标或服务商标权的许可。为避免歧义，除了正常的非独家、免版税的产品使用许可，任何形式的购买都不可被视为授予许可。对于任何违反保密义务、未经授权使用或以其他非法形式恶意使用所述文档和信息的违法侵权行为，移远通信有权追究法律责任。

商标

除另行规定，本文档中的任何内容均不授予在广告、宣传或其他方面使用移远通信或第三方的任何商标、商号及名称，或其缩略语，或其仿冒品的权利。

第三方权利

您理解本文档可能涉及一个或多个属于第三方的硬软件和文档（“第三方材料”）。您对此类第三方材料的使用应受本文档的所有限制和义务约束。

移远通信针对第三方材料不做任何明示或暗示的保证或陈述，包括但不限于任何暗示或法定的适销性或特定用途的适用性、平静受益权、系统集成、信息准确性以及与许可技术或被许可人使用许可技术相关的不侵犯任何第三方知识产权的保证。本协议中的任何内容都不构成移远通信对任何移远通信产品或任何其他硬件、设备、工具、信息或产品的开发、增强、修改、分销、营销、销售、提供销售或以其他方式维持生产的陈述或保证。此外，移远通信免除因交易过程、使用或贸易而产生的任何和所有保证。

隐私声明

为实现移远通信产品功能，特定设备数据将会上传至移远通信或第三方服务器（包括运营商、芯片供应商或您指定的服务器）。移远通信严格遵守相关法律法规，仅为实现产品功能之目的或在适用法律允许的情况下保留、使用、披露或以其他方式处理相关数据。当您与第三方进行数据交互前，请自行了解其隐私保护和数据安全政策。

免责声明

- 1) 移远通信不承担任何因未能遵守有关操作或设计规范而造成损害的责任。
- 2) 移远通信不承担因本文档中的任何因不准确、遗漏、或使用本文档中的信息而产生的任何责任。
- 3) 移远通信尽力确保开发中功能的完整性、准确性、及时性，但不排除上述功能错误或遗漏的可能。除非另有协议规定，否则移远通信对开发中功能的使用不做任何暗示或法定的保证。在适用法律允许的最大范围内，移远通信不对任何因使用开发中功能而遭受的损害承担责任，无论此类损害是否可以预见。
- 4) 移远通信对第三方网站及第三方资源的信息、内容、广告、商业报价、产品、服务和材料的可访问性、安全性、准确性、可用性、合法性和完整性不承担任何法律责任。

版权所有 ©上海移远通信技术股份有限公司 2022，保留一切权利。

Copyright © Quectel Wireless Solutions Co., Ltd. 2022.

文档历史

修订记录

版本	日期	作者	变更表述
-	2022-12-06	Jessie LEI/ Nick LIN	文档创建
1.0.0	2022-12-06	Jessie LEI/ Nick LIN	临时版本

目录

文档历史	3
目录	4
表格索引	6
图片索引	7
1 引言	8
2 A/B 系统升级概述	9
2.1. A/B 系统升级简介	9
2.2. A/B 系统启动流程	10
2.3. A/B 系统升级方案	11
2.4. A/B 系统分区损坏还原	11
3 调用 ql-lib-absys 和 ql-lib-fota 库升级	13
3.1. 升级准备	13
3.1.1. 升级步骤	13
3.1.2. 制作升级包镜像	13
3.1.2.1. 环境准备	13
3.1.2.2. 升级包镜像制作工具介绍	14
3.1.2.3. 制作升级包	14
3.1.2.4. 全量包制作方法	15
3.1.2.5. 单固件升级包制作方法	17
3.1.3. 下载升级包	18
3.2. 升级状态切换流程	18
3.3. API 调用推荐流程	20
3.4. ql-lib-fota 库相关 API	22
3.4.1. 头文件	22
3.4.2. 函数概览	22
3.4.3. 函数详解	22
3.4.3.1. ql_abfota_start_update	22
3.4.3.2. ql_abfota_get_update_status	23
3.5. ql-lib-absys 库相关 API	25
3.5.1. 头文件	25
3.5.2. 函数概览	25
3.5.3. 函数详解	26
3.5.3.1. ql_absys_getstatus	26
3.5.3.2. ql_absys_get_cur_active_part	27
3.5.3.3. ql_absys_sync	27
3.5.3.4. ql_absys_switch	28
3.6. 示例	29
3.6.1. 制作升级包	29
3.6.2. 编译测试程序	29
3.6.3. 下载升级包	29

3.6.4.	执行测试程序	29
3.6.5.	AB 系统非激活分区损坏查询及恢复处理	32
3.7.	资源消耗评估	34
3.7.1.	内存消耗评估	34
3.7.2.	Flash 空间消耗评估	35
3.7.3.	CPU 资源消耗评估	35
4	附录 参考文档及术语缩写	36

Preliminary Document
Not Checked

表格索引

表 1: 支持 A/B 系统升级方案的分区	9
表 2: <i>ql-lib-fota</i> 库函数概览	22
表 3: <i>ql-lib-absys</i> 库函数概览	25
表 4: 参考文档	36
表 5: 术语缩写	36

Preliminary Document
Not Checked

图片索引

图 1: A/B 系统 Flash 分区示意图	9
图 2: A/B 系统启动流程图	11
图 3: 调用 <i>ql-lib-absys</i> 和 <i>ql-lib-fota</i> 库升级方案下系统损坏还原流程	12
图 4: <i>fbf</i> 格式升级包制作工具及文件	14
图 5: 升级包制作指导	15
图 6: 制作全固件升级包的 <i>blf</i> 文件配置	16
图 7: 全量升级包	16
图 8: 制作全固件升级包的 <i>blf</i> 文件配置	17
图 9: 单固件升级包.....	18
图 10: 调用 <i>ql-lib-absys</i> 和 <i>ql-lib-fota</i> 库升级流程	19
图 11: <i>ql-lib-fota</i> 和 <i>ql-lib-absys</i> 库相关 API 调用推荐流程	21

1 引言

移远通信 EC200A-CN(TA)模块支持 QuecOpen®方案；QuecOpen®是开源的基于 Linux 的嵌入式开发平台，可简化 IoT 应用的软件设计和开发过程。有关 QuecOpen®的详细信息，请参考文档[1]。

本文档主要介绍 QuecOpen®方案下，移远通信 EC200A-CN(TA)模块支持的由移远通信自主研发的 A/B 系统升级方案，包括升级包的制作方法、升级流程以及注意事项等。

2 A/B 系统升级概述

2.1. A/B 系统升级简介

移远通信 EC200A-CN(TA) QuecOpen 模块使用 A/B 系统。模块将开机启动涉及的分区设置为两套分区，即 A 系统、B 系统，两个系统均烧录相同的镜像且相互备份。

A/B 系统的 Flash 分区示意图如下：

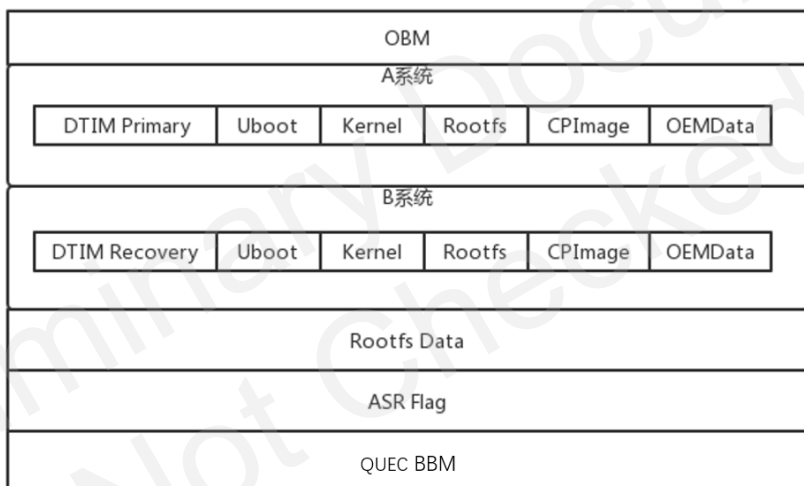


图 1: A/B 系统 Flash 分区示意图

如上图所示，Uboot、Kernel、Rootfs、CPImage 和 OEMData 均有两份（A 系统和 B 系统）。若其中一个系统损坏，模块切换至另一个系统运行，并还原损坏的系统。

表 1: 支持 A/B 系统升级方案的分区

No.	分区名称		更新频率
	A 系统	B 系统	
1	<i>oem_data-a</i>	<i>oem_data-b</i>	存放用户程序及配置，仅在升级用户程序及配置时更新。

2	<i>cpimage-a</i>	<i>cpimage-b</i>	移远通信更新固件基线或内容时，将对此分区进行更新。
3	<i>u-boot-a</i>	<i>u-boot-b</i>	客户进行分区修改调整时需要升级。
4	<i>kernel-a</i>	<i>kernel-b</i>	移远通信更新内容或用户进行自定义修改时更新。
5	<i>rootfs-a</i>	<i>rootfs-b</i>	移远通信更新内容或用户进行自定义修改时更新。

2.2. A/B 系统启动流程

系统上电后，OBM 从 ASR Flag 分区的固定位置读取当前活动系统的标志，如果该标志为“b”，则启动 B 系统，如果该标志为“a”，则启动 A 系统。

以启动 B 系统为例，OBM 首先加载 B 系统的 Uboot 分区，Uboot 分区启动后从 ASR Flag 分区的固定位置读取当前活动系统的标志。若该标志为“b”，则 Uboot 分区从 B 系统的 DTIM 分区里解析出 Kernel、Rootfs 和 CPImage 的 Flash 地址，然后校验对应的 Image 并加载。模块会在启动参数里加入“system=b”并传递给 Kernel，同时将 Rootfs 对应的 MTD 设备传递给 Kernel 分区并启动，Kernel 分区再挂载 Rootfs 分区。最后加载 OEM_Data 分区，因启动参数里包含了“system=b”字符串，所以挂载 B 系统的 OEM_Data 分区。

A 系统和 B 系统里的每个镜像在 Kernel 设备树里都有对应的分区，每个分区对应一个 MTD 设备（/dev/mtdblockx）。

A/B 系统启动流程示意图如下：

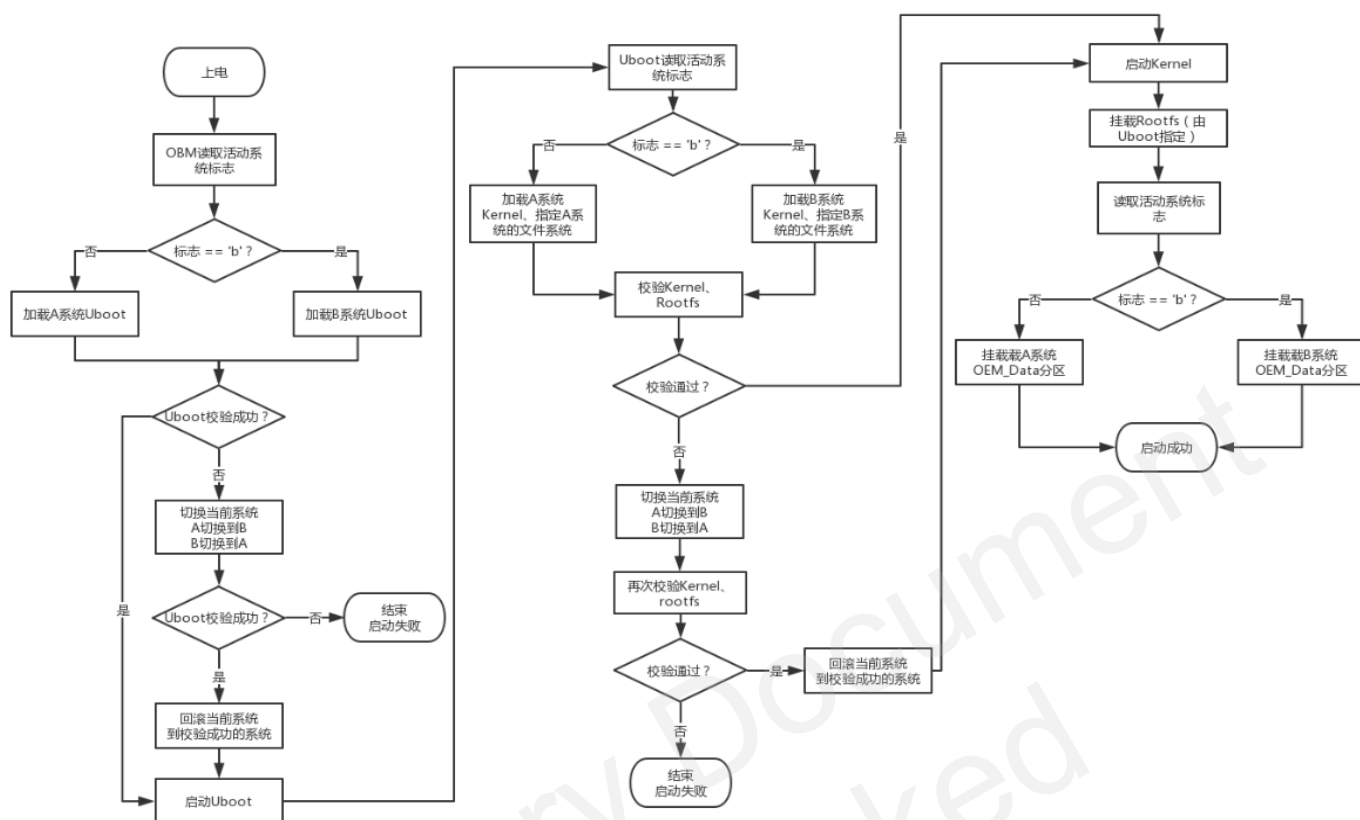


图 2: A/B 系统启动流程图

备注

模块开机时启动的系统为激活系统，未启动的系统为非激活系统。升级过程中仅会升级未激活系统，当前正在运行的系统不允许升级。

2.3. A/B 系统升级方案

移远通信 EC200A-CN(TA) QuecOpen 模块通过调用 *ql-lib-absys* 和 *ql-lib-fota* 库的相关 API 触发和控制 A/B 系统升级。升级包格式为。详见第3章。

2.4. A/B 系统分区损坏还原

模块支持在分区损坏时，切换 A/B 系统并还原损坏分区。例如，模块加载 A 系统失败时，自动切换至 B 系统上运行，同时标记 A 系统为损坏分区；B 系统加载成功后检查到 A 系统有损坏标记，将对 A 系统进行还原。若 A、B 系统同镜像分区（如 *kernel-a* 和 *kernel-b*）同时损坏，模块启动失败，等待重新烧录固件。

若开机时出现分区损坏，模块会自动切换系统进行加载，并标记 *is_damaged* 字段为 *TRUE*。用户应用程序需主动调用 *ql_absys_getstatus()* 检测非激活系统是否损坏，若损坏，需主动调用 *ql_absys_sync()* 同步系统以还原损坏分区。还原完成后模块自动清除损坏标记，此时 *is_damaged* 字段为 *FALSE*。

调用 *ql-lib-absys* 和 *ql-lib-fota* 库升级方案系统损坏还原流程如下：

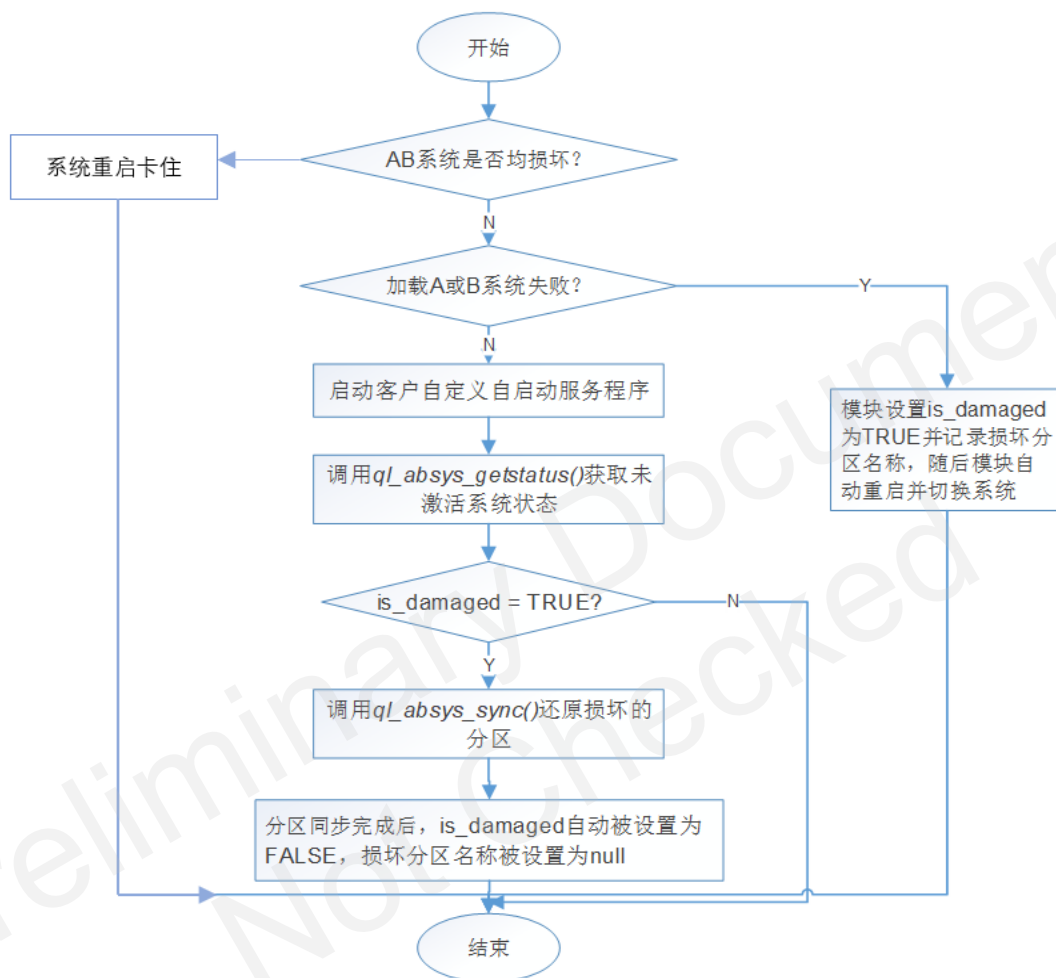


图 3：调用 *ql-lib-absys* 和 *ql-lib-fota* 库升级方案下系统损坏还原流程

3 调用 ql-lib-absys 和 ql-lib-fota 库升级

本章介绍通过调用 *ql-lib-absys* 和 *ql-lib-fota* 库的相关 API 控制 A/B 系统升级的方法。

备注

1. EC200A-CN(TA)模块默认通过调用 *ql-lib-absys* 和 *ql-lib-fota* 库的相关 API 触发 A/B 系统升级，并通过 QuecOpen SDK 中 *ql-ol-rootfs/usr/bin/ota* 服务程序管理 A/B 系统升级以及分区同步。
2. 模块支持通过调用 *ql-lib-absys* 和 *ql-lib-fota* 库相关 API 控制并管理 A/B 系统升级。调用 *ql-lib-fota* 库升级时，默认移除模块自带的自启动服务程序 *ql-ol-rootfs/usr/bin/ota*，将所有的操作权交给用户。
3. 模块中的 A/B 系统互为备份，调用 *ql-lib-absys* 库相关 API 控制和管理 A/B 系统升级并移除自启动服务程序 *ql-ol-rootfs/usr/bin/ota* 时，分区自动还原功能也相应被移除，因此用户需添加自定义的自启动服务程序。用户应用程序调用 *ql_absys_getstatus()* 检查非激活系统是否有损坏，若有损坏，则需用户应用程序调用 *ql_absys_sync()* 还原损坏的系统。详见第 3.6.5 章。

3.1. 升级准备

本节以 EC200ACNTAR02A06M2G_OCPU 固件版本为例介绍通过调用 *ql-lib-absys* 和 *ql-lib-fota* 库进行 A/B 系统升级的准备工作

3.1.1. 升级步骤

A/B 系统升级步骤如下：

步骤 1：制作升级包镜像 *quectel_AB_OTA.img*，详见第 3.1.2 章。

步骤 2：将制作后的升级镜像及升级程序 *test_absys* 下载到模块的指定目录，一般为内部 Flash 分区或 eMMC 分区。

步骤 3：使用升级程序 *test_absys* 触发升级：

3.1.2. 制作升级包镜像

3.1.2.1. 环境准备

需在 Linux 环境下准备：

制作脚本: `ql-ol-extsdk_ec200acntar02a06m2g_ocpu/tools/fota/gen_fota_image.sh`

制作升级包指导: `ql-ol-extsdk_ec200acntar02a06m2g_ocpu/tools/fota/Readme.txt`

可参考 `Readme.txt` 文件初步了解如何制作升级包镜像。

3.1.2.2. 升级包镜像制作工具介绍

1. 所需文件:

<code>quectel_skylark_pm802_standard_AB.blf</code>	非工厂配置 blf 文件
<code>quectel_AB_OTA.blf</code>	OTA 配置 blf 文件
<code>mversion</code>	FOTA 镜像版本号

2. 所需工具:

<code>tim_builder</code>	用于编译待升级的固件并生成新的 TIM 文件
<code>mkotafbf</code>	用于制作 <code>fbf</code> 格式的升级包镜像文件

3. `mkotafbf` 工具常用命令描述:

- f: 输入 blf 文件名
- o: 输出 fbf 文件名
- v: FOTA 镜像版本号 (mversion)

```

ol@ol:~/workspace/ASR_ADK/ql-ol-extsdk_ec200acntar02a06m2g_ocpu/tools/fota$ tree
.
├── gen_fota_image.sh
├── mkotafbf
├── mversion
├── part_resizing.sh
├── quectel_AB_OTA.blf
├── quectel_skylark_pm802_standard_AB.blf
├── Readme.txt
├── target
└── tim_builder

1 directory, 8 files
  
```

图 4: `fbf` 格式升级包制作工具及文件

3.1.2.3. 制作升级包

升级包制作指导详见 `ql-ol-extsdk_ec200acntar02a06m2g_ocpu/tools/fota/Readme.txt`。

```
How to generate fota image file for ASR platform OTA upgrade

Introduction:
gen_fota_image.sh will generate a fota image file for OTA upgrade on ASR platform.

upgrade firmware:
copy need OTA upgrade firmware : TLoader_QSPINAND.bin, u-boot, kernel, rootfs, tos.bin, oem_data, cp(ARBEL.bin,MSA.bin,RFPLUGIN.bin) to sdk tools/fota/

Usage: ./gen_fota_image.sh (part_name)

For example:
Full Firmware Upgrade
./gen_fota_image.sh

Single Firmware Upgrade
Upgrade the single firmware uboot
./gen_fota_image.sh uboot

Upgrade the single firmware zimage
./gen_fota_image.sh zimage

Upgrade the single firmware rootfs
./gen_fota_image.sh rootfs

Upgrade the single firmware oemapp
./gen_fota_image.sh oemapp

Upgrade the single firmware cpimage
./gen_fota_image.sh cpimage
```

图 5：升级包制作指导

备注

在制作 **fbf** 升级包镜像时，除了需要升级的固件外，还需拷贝 **TLoader_QSPINAND.bin** 文件到 **/fota** 目录下，以编译进 **TIM** 镜像并打包进 **fbf** 升级包。

模块支持全量包升级和单固件升级，升级包制作方法如下：

在 **Ubuntu** 终端进入 **ql-ol-extsdk_ec200acntar02a06m2g_ocpu/tools/fota** 目录，使用 **chmod** 赋予所有脚本及工具可执行权。

3.1.2.4. 全量包制作方法

1. 拷贝需要升级的固件及 **TLoader_QSPINAND.bin** 到 **/fota** 目录；
2. **blf** 文件配置：

```
[Image_List]
1_Image_Enable = 1
1_Image_Tim_Included = 2
1_Image_Image_ID = 0x41524249
1_Image_Next_Image_ID = 0x47524249
1_Image_Path = ARBEL.bin
1_Image_Flash_Entry_Address = 0x00580000
1_Image_Load_Address = 0xFFFFFFFF
1_Image_Type = RAW
1_Image_ID_Name = ARBI
1_Image_Erase_Size =
1_Image_Partition_Number = 0
1_Image_Hash_Algorithm_ID = SHA-256
1_Image_Image_Size_To_Hash_in_bytes = 0xFFFFFFFF
```

```
16_Image_Enable = 1
16_Image_Tim_Included = 3
16_Image_Image_ID = 0x4F454D44
16_Image_Next_Image_ID = 0xFFFFFFFF
16_Image_Path = oem_data.ubi
16_Image_Flash_Entry_Address = 0x06D00000
16_Image_Load_Address = 0xFFFFFFFF
16_Image_Type = RAW
16_Image_ID_Name = OEMD
16_Image_Erase_Size = 0x01400000
16_Image_Partition_Number = 0
16_Image_Hash_Algorithm_ID = SHA-256
16_Image_Image_Size_To_Hash_in_bytes = 0xFFFFFFFF
```

图 6：制作全固件升级包的 blf 文件配置

备注

SDK 中的 blf 文件默认配置为全包固件升级。

3. 执行如下命令开始制作全量升级包：

```
./gen_fota_image.sh
```

4. 命令执行成功后，生成的全量升级包 `quectel_AB_OTA.img` 将存放在 `ql-ol-extsdk_ec200acntar02a06m2g_ocpu/tools/fota` 目录下：

```
makeFOTAImage for MCB successfully...
ol@ol:~/workspace/ASR_ADK/ql-ol-extsdk_ec200acntar02a06m2g_ocpu/tools/fota$ tree
.
├── ARBEL.bin
├── DDR_ESMT_64M0xFFFF_DTIm.PPsetting
├── DDR_ESMT_64M0xFFFF_tim_falcon_qspinand.bin
├── DDR_WINBOND_128M0xFFFF_DTIm.PPsetting
├── DDR_WINBOND_128M0xFFFF_tim_falcon_qspinand.bin
├── DDR_ZMOS_64M0xFFFF_DTIm.PPsetting
├── DDR_ZMOS_64M0xFFFF_tim_falcon_qspinand.bin
├── DKB_timheader.bin
├── DTIm.PPsetting
├── DTIm.PPsetting2
├── DTIm.Primary
├── DTIm.Recovery
├── gen_fota_image.sh
├── mkotaafb
├── MSA.bin
├── mversion
├── oem_data.ubi
├── part_resizing.sh
├── quectel_AB_OTA.blf
├── quectel_AB_OTA.img
├── quectel_skylark_pm802_standard_AB.blf
├── Readme.txt
├── RFPLUGIN.bin
├── root.squashfs
├── target
├── tim_builder
├── tim_falcon_qspinand.bin
├── TLoader_QSPINAND.bin
├── tos.bin
├── u-boot.bin
├── zImage
└── 1 directory, 30 files
```

图 7：全量升级包

3.1.2.5. 单固件升级包制作方法

以单固件升级 oemapp 为例。

备注

模块支持单固件升级，但需要维护好模块内当前的固件版本，即除了升级的单固件外，还需要将模块内同版本的其他固件拷贝至/fota 目录；例如升级 oemapp 单固件，需要将模块内同版本的 zImage、uboot、rootfs、cpimage 固件拷贝到/fota 目录，再制作 oemapp 单固件升级包。

1. 拷贝需要升级的固件及 TLoder_QSPINAND.bin 到/fota 目录下；
2. blf 文件配置被自动修改如下：

```
[Image_List]
8_Image_Enable = 1
8_Image_Tim_Included = 2
8_Image_Image_ID = 0x4F454D44
8_Image_Next_Image_ID = 0x41524249
8_Image_Path = oem_data.ubi
8_Image_Flash_Entry_Address = 0x02D00000
8_Image_Load_Address = 0xFFFFFFFF
8_Image_Type = RAW
8_Image_ID_Name = OEMD
8_Image_Erase_Size = 0x01400000
8_Image_Partition_Number = 0
8_Image_Hash_Algorithm_ID = SHA-256
8_Image_Image_Size_To_Hash_in_bytes = 0xFFFFFFFF
16_Image_Enable = 1
16_Image_Tim_Included = 3
16_Image_Image_ID = 0x4F454D44
16_Image_Next_Image_ID = 0xFFFFFFFF
16_Image_Path = oem_data.ubi
16_Image_Flash_Entry_Address = 0x06D00000
16_Image_Load_Address = 0xFFFFFFFF
16_Image_Type = RAW
16_Image_ID_Name = OEMD
16_Image_Erase_Size = 0x01400000
16_Image_Partition_Number = 0
16_Image_Hash_Algorithm_ID = SHA-256
16_Image_Image_Size_To_Hash_in_bytes = 0xFFFFFFFF
```

图 8：制作全固件升级包的 blf 文件配置

备注

SDK 包中的 blf 文件配置为单固件升级时只需保留目标固件的 image list 信息。

3. 执行如下命令开始制作单固件升级包：

```
./gen_fota_image.sh oemapp
```

- 命令执行成功后，生成的单固件升级包 *quectel_AB_OTA.img* 将存放在 *ql-ol-extsdk_ec200acntar02a06m2g_ocpu/tools/fota* 目录下。

```
ol@ol:~/workspace/ASR_ADK/ql-ol-extsdk_ec200acntar02a06m2g_ocpu/tools/fota$ tree
.
├── ARBEL.bin
├── DDR_ESMT_64M0xFFFF_DTIm.PPsetting
├── DDR_ESMT_64M0xFFFF_tim_falcon_qspinand.bin
├── DDR_WINBOND_128M0xFFFF_DTIm.PPsetting
├── DDR_WINBOND_128M0xFFFF_tim_falcon_qspinand.bin
├── DDR_ZMOS_64M0xFFFF_DTIm.PPsetting
├── DDR_ZMOS_64M0xFFFF_tim_falcon_qspinand.bin
├── DKB_timheader.bin
├── DTIm.PPsetting
├── DTIm.PPsetting2
├── DTIm.Primary
├── DTIm.Recovery
├── gen_fota_image.sh
├── mkotaafb
├── MSA.bin
├── mversion
├── oem_data.ubi
├── part_resizing.sh
├── quectel_AB_OTA.blf
├── quectel_AB_OTA.img
├── quectel_skylark_pm802_standard_AB.blf
├── Readme.txt
├── RFPLUGIN.bin
├── root.squashfs
├── target
├── tim_builder
├── tim_falcon_qspinand.bin
├── TLoader_QSPINAND.bin
├── tos.bin
├── u-boot.bin
└── zImage

1 directory, 30 files
```

图 9：单固件升级包

3.1.3. 下载升级包

将生成的升级包 *quectel_AB_OTA.img* 下载到模块中指定目录，一般存放至 Flash 分区或者 eMMC 分区。无论下载至何分区，均需保证该分区开机自动挂载，否则重新上电会因无法找到升级包从而导致报错。

3.2. 升级状态切换流程

调用 *ql-lib-absys* 和 *ql-lib-fota* 库控制 A/B 系统升级，升级状态共有 6 种：SUCCEED、UPDATE、WRITEDONE、NEEDSYNC、BACKUP、FAILED。升级状态切换流程如下：

- 当前可进行升级操作。此时升级状态为 SUCCEED，且系统初始状态为 SUCCEED。

2. 正在升级非激活系统。此时升级状态为 UPDATE。
3. 非激活系统升级完成。此时升级状态为 WRITEDONE。
4. 执行系统切换。切换成功后，升级状态为 NEEDSYNC。
5. 正在同步分区。此时升级状态为 BACKUP。
6. 分区同步完成。此时升级状态为 SUCCEED。

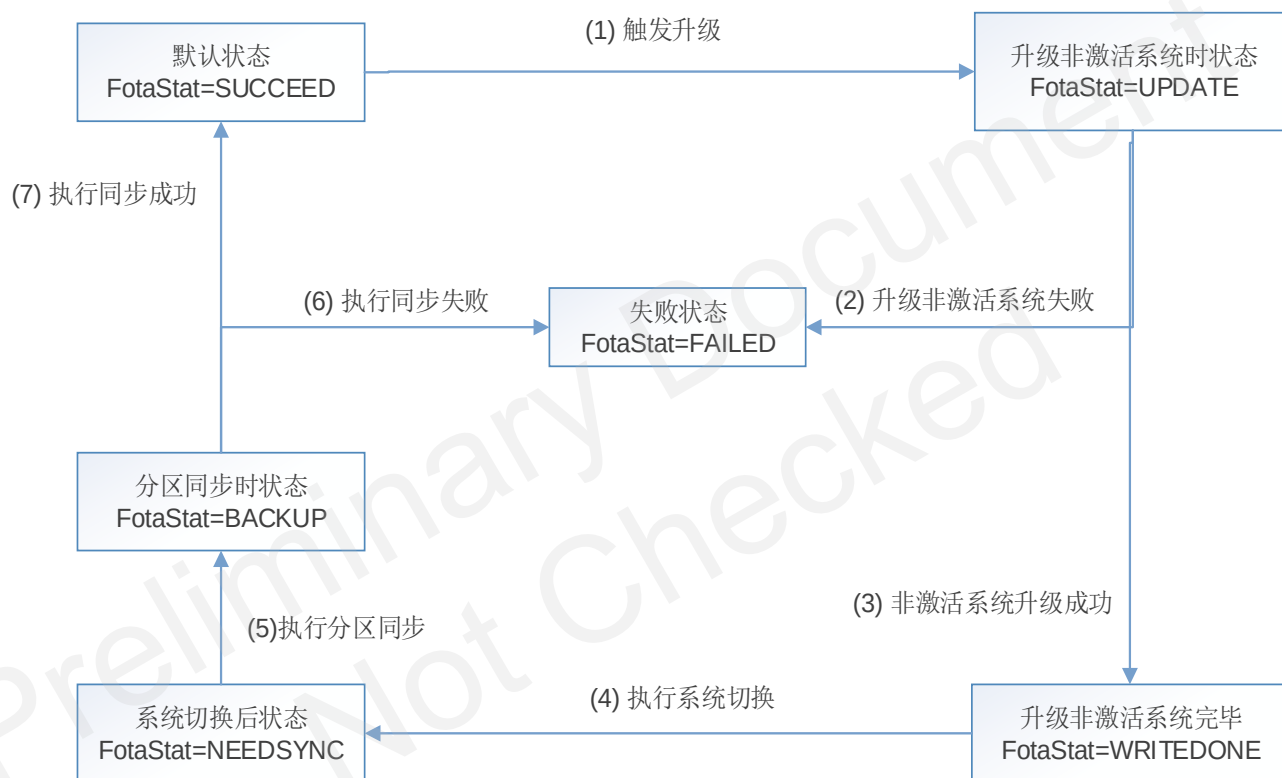


图 10: 调用 *ql-lib-absys* 和 *ql-lib-fota* 库升级流程

(1) 触发升级

调用 *ql_abfota_start_update()* 启动升级，升级状态切换为 UPDATE，表示当前正在升级非激活系统；

(2) 升级非激活系统失败

若升级过程中出现异常，升级状态自动切换为 FAILED，表示异常退出 A/B 系统升级程序；

(3) 升级非激活系统成功

升级非激活系统成功结束后，升级状态自动切换为 WRITEDONE，表示升级非激活系统成功；

（4）执行系统切换

调用 `ql_absys_switch()` 切换系统。函数执行成功后自动重启模块，模块将切换到升级后的系统上运行（即新固件），升级状态自动切换为 **NEEDSYNC**，表示 A、B 系统不一致，需同步 A、B 系统；

（5）执行分区同步

调用 `ql_absys_sync()` 同步分区时，升级状态自动切换为 **BACKUP**，表示正在同步分区；

（6）执行同步失败

若 A、B 系统同步失败，升级状态自动切换为 **FAILED**，表示当前非激活系统被破坏，需进行还原；

（7）执行同步成功

若 A、B 系统同步成功，升级状态自动切换为 **SUCCEED**，表示升级完成且 A、B 系统已同步。

3.3. API 调用推荐流程

调用 `ql-lib-fota` 和 `ql-lib-absys` 库相关 API 进行 A/B 系统升级时，API 调用推荐流程如下图所示。

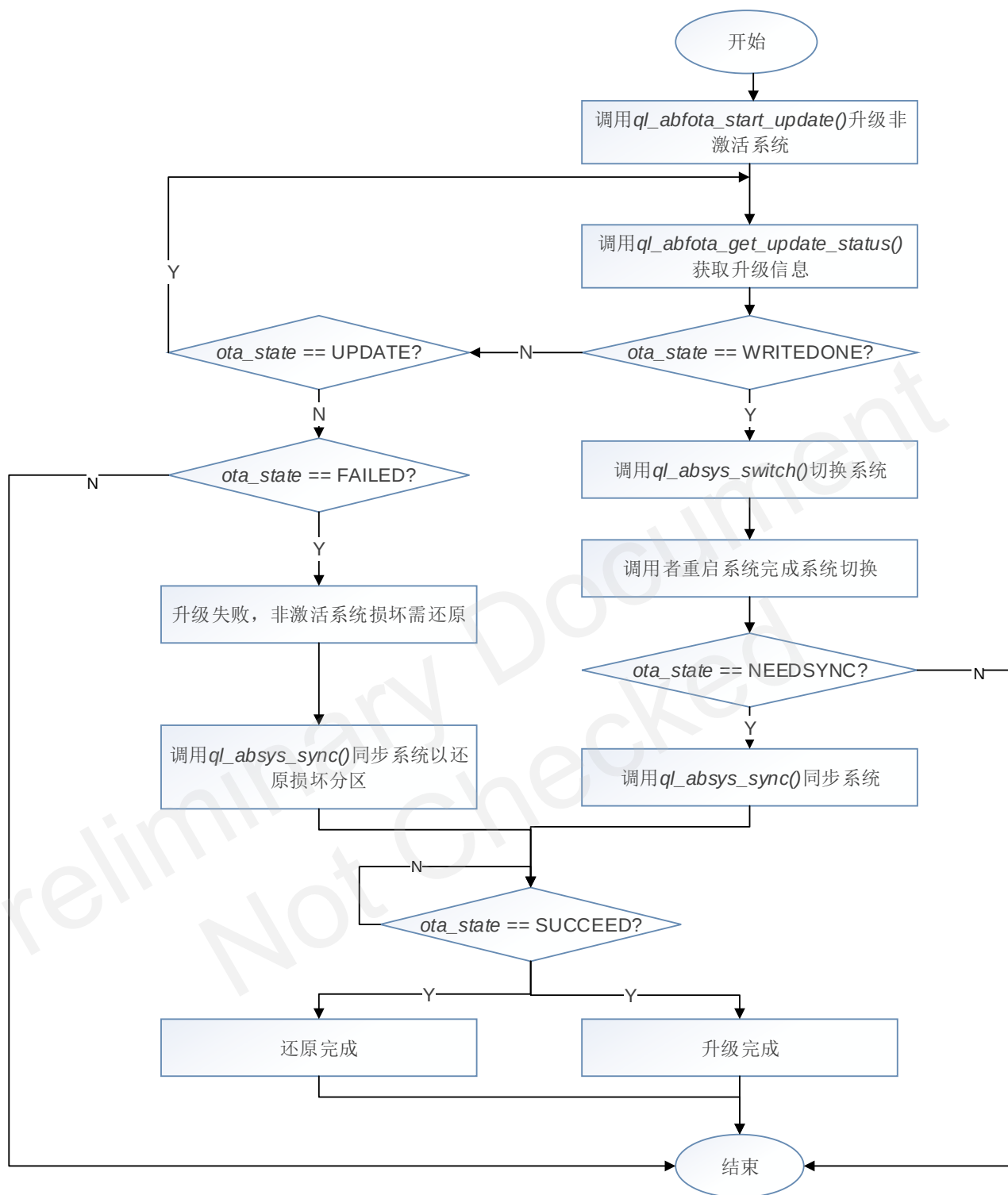


图 11: *ql-lib-fota* 和 *ql-lib-absys* 库相关 API 调用推荐流程

备注

1. 若非激活系统升级失败，非激活系统固件被损坏，升级状态切换为 **FAILED**。如需继续升级，可直接再次升级非激活系统；如无需升级，调用 `ql_absys_sync()` 进行还原即可。

2. 无论升级成功或失败，模块均不会删除升级包，升级包下载/删除由用户应用程序管理。
3. 升级状态切换到 **NEEDSYNC** 时，说明模块已经启动并在已升级系统上运行，此时可无需等待系统同步完成即可上报升级结果至客户应用程序，但分区同步必须要执行。

3.4. ql-lib-fota 库相关 API

3.4.1. 头文件

头文件为 *ql_fota_api.h* 和 *fota_info.h*，位于 SDK 包的 *ql-sysroots\usr\include* 目录下。若无特别说明，本文档所涉及头文件均在该目录下。

3.4.2. 函数概览

表 2: *ql-lib-fota* 库函数概览

函数	说明
<i>ql_abfota_start_update()</i>	触发 A/B 系统升级
<i>ql_abfota_get_update_status()</i>	获取模块当前升级信息

3.4.3. 函数详解

3.4.3.1. ql_abfota_start_update

该函数用于触发 A/B 系统升级。

- 函数原型

```
int ql_abfota_start_update(const char *package_path)
```

- 参数

package_path:

[In] 指针，指向升级包所在路径。升级包的名称为 *quectel_AB_OTA.img*。例如：升级包存储在 */mnt/sdcard/* 目录下，则该参数值为 */mnt/sdcard/quectel_AB_OTA.img*。

- 返回值

- 0 函数执行成功
- 1 函数执行失败，传入参数无效，或者设置升级起始状态失败。
- 其他值 升级失败，详见第3.4.3.1.1章。

3.4.3.1.1. fota_exit_code_t

异常退出码枚举定义如下：

```
typedef enum
{
    E_NO_ERROR=0,
    E_FOTA_INIT_FAILED=1,
    E_UPDATE_PACKAGE_NOEXIST=2,
    E_WRITE_SEG_FAILED=3,
} fota_exit_code_t
```

- 成员

成员	描述
<i>E_NO_ERROR</i>	升级成功
<i>E_FOTA_INIT_FAILED</i>	升级初始化失败
<i>E_UPDATE_PACKAGE_NOEXIST</i>	升级包不存在
<i>E_WRITE_SEG_FAILED</i>	下载段数据失败

3.4.3.2. ql_abfota_get_update_status

该函数用于获取模块当前升级信息。例如：升级状态、升级进度，以及升级异常退出码。

- 函数原型

```
int ql_abfota_get_update_status(update_info_t *update_info)
```

- 参数

update_info:

[In/Out] 指向存储当前升级状态、升级进度、以及升级异常退出码的内存地址；详见第3.4.3.2.1章。

- 返回值
- 0 函数执行成功
- 1 函数执行失败

备注

当调用该函数获取的升级状态为**FAILED**且退出码不为0时，说明升级失败，此时若需继续升级，调用 *ql_abfota_start_update()*即可恢复正常升级状态。

3.4.3.2.1. update_info_t

升级信息结构体信息定义如下：

```
typedef struct
{
    unsigned int percentage;
    fota_state_t ota_state;
    fota_exit_code_t exit_code;
} update_info_t
```

- 参数

类型	参数	描述
<i>unsigned int</i>	<i>percentage</i>	升级的进度百分比。范围：0~100。
<i>fota_state_t</i>	<i>ota_state</i>	升级状态；详见第3.4.3.2.2章。
<i>fota_exit_code_t</i>	<i>exit_code</i>	升级异常退出码；详见第3.4.3.1.1章。

3.4.3.2.2. fota_state_t

升级状态枚举定义如下。该信息存储在 Flash 中，掉电不丢失。

```
typedef enum
{
    SUCCEED = 0,
    UPDATE,
    BACKUP,
    FAILED,
    WRITEDONE,
    NEEDSYNC,
    UNKNOWN_STATUS
} fota_state_t
```

● 成员

成员	描述
<i>SUCCEED</i>	默认状态。升级成功，并且 A/B 系统同步成功。
<i>UPDATE</i>	正在升级非激活系统
<i>BACKUP</i>	正在同步非激活系统
<i>FAILED</i>	升级失败
<i>WRITEDONE</i>	升级非激活系统成功
<i>NEEDSYNC</i>	已切换运行系统，尚未同步系统。
<i>UNKNOWN_STATUS</i>	未知状态，代码中用于边界检查。

3.5. ql-lib-absys 库相关 API

3.5.1. 头文件

头文件为 *ql_absys.h* 和 *fota_info.h*，位于 SDK 包的 *ql-sysroots\usr\include* 目录下。若无特别说明，本文档所涉及头文件均在该目录下。

3.5.2. 函数概览

表 3: *ql-lib-absys* 库函数概览

函数	说明
<i>ql_absys_getstatus()</i>	查询非激活系统状态
<i>ql_absys_get_cur_active_part()</i>	查询当前运行系统
<i>ql_absys_sync()</i>	同步未激活系统
<i>ql_absys_switch()</i>	切换待运行系统

3.5.3. 函数详解

3.5.3.1. ql_absys_getstatus

该函数用于查询非激活系统状态。

● 函数原型

```
int ql_absys_getstatus(sysstatus_t *sys_state)
```

● 参数

sys_state:
[Out] 指针，指向存放非激活系统状态的结构体；详见第3.5.3.1.1章。

● 返回值

- 0 函数执行成功
- 1 函数执行失败

3.5.3.1.1. sysstatus_t

非激活系统状态结构体定义如下。状态信息存储在 Flash 中，掉电不会丢失。

```
typedef struct
{
    fota_state_t    ota_state;
    bool            is_damaged;
    unsigned char   damaged_partname[16]
} sysstatus_t
```

● 参数

类型	参数	描述
fota_state_t	ota_state	当前非激活系统升级状态；详见第3.4.3.2.2章。
bool	is_damaged	非激活系统是否损坏
unsigned char	damaged_partname	非激活系统发生损坏的分区名称

3.5.3.2. ql_absys_get_cur_active_part

该函数用于查询当前运行的系统。

- 函数原型

```
int ql_absys_get_cur_active_part(absystem_t *cur_system)
```

- 参数

cur_system:

[In/Out] 指针，指向用于存储读取到的信息枚举；详见第3.5.3.2.1章。

- 返回值

0 函数执行成功

-1 函数执行失败

3.5.3.2.1. absystem_t

A/B 系统枚举定义如下：

```
typedef enum absystem
{
    SYSTEM_A=0,
    SYSTEM_B=1
} absystem_t
```

- 成员

成员	描述
SYSTEM_A	A 系统
SYSTEM_B	B 系统

3.5.3.3. ql_absys_sync

该函数用于同步非激活系统，将当前运行系统的镜像文件拷贝并覆盖至非激活系统。调用该函数后升级状态切换为 **BACKUP**；同步成功后升级状态切换为 **SUCCEED**，同时清除分区损坏标记（即 *is_damaged* = *FALSE*）以及损坏分区名称记录。

- 函数原型

```
int ql_absys_sync(void)
```

- 参数

无

- 返回值

0 函数执行成功

-1 函数执行失败

3.5.3.4. ql_absys_switch

该函数用于切换 A/B 系统。成功执行该函数设置完系统标记后，自动重启模块实现系统切换。

1. 若升级状态为 **SUCCEED**，可切换系统；
2. 若升级状态为 **WRITEDONE**，表示非激活系统已升级完成，可进行系统切换；切换过程中模块将自动重启且升级状态更新为 **NEEDSYNC**；
3. 若升级状态为 **UPDATE** 或者 **BACKUP**，表示正在升级非激活系统或者正在同步系统，不可切换系统；若进行系统切换，升级会失败并可能造成其他错误；
4. 若升级状态为 **FAILED**，表示非激活系统被损坏（升级非激活系统失败或同步非激活系统失败），不可切换系统；若进行系统切换，切换后检测到系统损坏会再次切换回来；
5. 若 **is_damaged** 是 **TRUE**，表示非激活系统有分区损坏，不可切换系统；若进行系统切换，切换后检测到系统损坏会再次切换回来，可调用 **ql_absys_getstatus()** 查询损坏分区信息。

- 函数原型

```
int ql_absys_switch(void)
```

- 参数

无

- 返回值

0 函数执行成功，等待模块重启

-1 获取或者设置状态时出错

-2 非激活系统正在被升级或者正在被同步，不能切换

-3 非激活系统升级时被破坏或者同步时被破坏，不能切换

3.6. 示例

QuecOpen SDK 中提供函数测试示例 *sample/absys/test_absys*，本章节介绍如何使用该测试程序进行 A/B 系统升级。

3.6.1. 制作升级包

请参考第 3.1.2 章制作升级包。

3.6.2. 编译测试程序

在 Ubuntu 终端窗口进入 QuecOpen SDK 的 *sample/absys/* 目录，执行 **make** 开始编译。编译完成后，生成可执行文件 *test_absys* 并存储在当前目录下。

```
ol@ol:~/workspace/ASR_ADK/ql-ol-extsdk_ec200acntar02a06m2g_ocpu/sample/absys$ make
/opt/ql_crosstools/ql-ag35-1803e-gcc-8.4.0-v1-toolchain/bin/arm-openwrt-linux-gcc -o test_absys.o -c test_absys.c -march=armv7-a -marm -mfpu=neon -mfloat-abi=hard -I. -I/home/ol/workspace/ASR_ADK/ql-ol-extsdk_ec200acntar02a06m2g_ocpu/ql-sysroots/usr/include -I/home/ol/workspace/ASR_ADK/ql-ol-extsdk_ec200acntar02a06m2g_ocpu/ql-sysroots/usr/include/ql-sdk -Wall -Wundef
/opt/ql_crosstools/ql-ag35-1803e-gcc-8.4.0-v1-toolchain/bin/arm-openwrt-linux-gcc -o test_absys test_absys.o -march=armv7-a -marm -mfpu=neon -mfloat-abi=hard -L. -L/opt/ql_crosstools/ql-ag35-1803e-gcc-8.4.0-v1-toolchain/bin/./usr/lib -L/opt/ql_crosstools/ql-ag35-1803e-gcc-8.4.0-v1-toolchain/bin/./lib -L/home/ol/workspace/ASR_ADK/ql-ol-extsdk_ec200acntar02a06m2g_ocpu/ql-sysroots/lib -L/home/ol/workspace/ASR_ADK/ql-ol-extsdk_ec200acntar02a06m2g_ocpu/ql-sysroots/usr/lib -lql_l1b_absys -lql_l1b_fota -lubox -lucil -lblobmsg_json -ljson-c -ljson_script -lrflutil -llog -lprop2uci -lintel
ol@ol:~/workspace/ASR_ADK/ql-ol-extsdk_ec200acntar02a06m2g_ocpu/sample/absys$ ls
Makefile  test_absys  test_absys.c  test_absys.o
ol@ol:~/workspace/ASR_ADK/ql-ol-extsdk_ec200acntar02a06m2g_ocpu/sample/absys$
```

备注

测试用例中 *sample/fota/* 目录下的 *test_fota* 只提供了 *ql_abfota_start_update()* 和 *ql_abfota_get_update_status()* 两个 API 的测试，*test_absys* 提供了 A/B 系统升级所有函数的测试，因此用户只需参考使用 *test_absys*。

3.6.3. 下载升级包

将制作完成的升级包镜像 *quectel_AB_OTA.img* 下载至模块的可访问分区 (rootfs_data、外挂 eMMC)；将编译生成的可执行文件 *test_absys* 上传至模块的可读写分区 *rootfs_data*。

```
D:\platform-tools>adb push Z:\quectel_AB_OTA.img /data
* daemon not running. starting it now at tcp:5037 *
* daemon started successfully *
Z:\quectel_AB_OTA.img: 1 file pushed. 1.8 MB/s (31318016 bytes in 16.215s)

D:\platform-tools>adb push Z:\test_absys /data
Z:\test_absys: 1 file pushed. 0.8 MB/s (15740 bytes in 0.020s)
```

3.6.4. 执行测试程序

赋予测试程序 *test_absys* 可执行权限并运行。

1. 运行程序后根据提示输入“0”，再输入升级包所在路径/data/quectel_AB_OTA.img 或者 /mnt/sdcard/quectel_AB_OTA.img，回车开始升级非激活系统。

```

root@OpenWrt:/data# cd /
root@OpenWrt:/# ls
NVM      dev      init      mnt      rom      sdcard    usr
bin      etc      lib      overlay  root      sys      var
data     gnssfirm log      proc     sbin     tmp       www
root@OpenWrt:/# cd /data/
root@OpenWrt:/data# ls
Fs      mmcblk0p2      usb
eeh_journal.txt  quectel_AB_OTA.img
mmcblk0p1      test_absys
root@OpenWrt:/data# chmod 777 test_absys quectel_AB_OTA.img
root@OpenWrt:/data# ./test_absys
Quectel OTA API test sample, version : v0.0.1

0      API : ql_abfota_start_update
1      API : ql_abfota_get_update_status
2      API : ql_absys_switch
3      API : ql_absys_sync
4      API : ql_absys_get_cur_active_part
5      API : ql_absys_getstatus
-1     exit
Please enter your choice: 1

[DBG][test_get_fota_upgrade_info_172][358] Current fota progress: 0
[DBG][test_get_fota_upgrade_info_173][358] Current fota state: SUCCEED
[DBG][test_get_fota_upgrade_info_174][358] Current fota exit code: 0
Please enter your choice: 0

please input the fota fbf package file: /data/quectel_AB_OTA.img
status: 0, percent: 10%
status: 0, percent: 20%
status: 0, percent: 30%
status: 0, percent: 40%
status: 0, percent: 50%
status: 0, percent: 60%
status: 0, percent: 70%
status: 0, percent: 80%
status: 0, percent: 90%
status: 0, percent: 100%
download firmware success!
[DBG][test_ota_api_start_105][443] Update in-active partition SUCCEED

```

2. 升级非激活系统成功后，显示“update in-active partition SUCCEED”，随后根据提示输入“1”。回车返回升级的状态信息，其中包括升级进度百分比、升级状态以及退出码。

```

status: 0, percent: 40%
status: 0, percent: 50%
status: 0, percent: 60%
status: 0, percent: 70%
status: 0, percent: 80%
status: 0, percent: 90%
status: 0, percent: 100%
download firmware success!
[DBG][test_ota_api_start_105][443] Update in-active partition SUCCEED
Please enter your choice: 1

[DBG][test_get_fota_upgrade_info_172][481] Current fota progress: 100
[DBG][test_get_fota_upgrade_info_173][481] Current fota state: WRITEDONE
[DBG][test_get_fota_upgrade_info_174][481] Current fota exit code: 0
Please enter your choice:

```

3. 根据提示输入“4”查询当前运行的系统。

```

root@openwrt:/data# ./test_absys
Quectel OTA API test sample, version : v0.0.1

0      API : ql_abfota_start_update
1      API : ql_abfota_get_update_status
2      API : ql_absys_switch
3      API : ql_absys_sync
4      API : ql_absys_get_cur_active_part
5      API : ql_absys_getstatus
-1     exit
Please enter your choice: 4

[DBG][test_ql_absys_get_cur_active_part_222][550] Current active part is A
Please enter your choice:

```

4. 根据提示输入“2”进行系统切换。输入“2”并回车后，模块将自动重启并切换至已升级系统运行。

```

root@openwrt:/data# ./test_absys
Quectel OTA API test sample, version : v0.0.1

0      API : ql_abfota_start_update
1      API : ql_abfota_get_update_status
2      API : ql_absys_switch
3      API : ql_absys_sync
4      API : ql_absys_get_cur_active_part
5      API : ql_absys_getstatus
-1     exit
Please enter your choice: 4

[DBG][test_ql_absys_get_cur_active_part_222][550] Current active part is A
Please enter your choice: 2

[DBG][test_ql_absys_switch_188][577] It is okay to swith AB part to run
[ 441.870893] procd killer:reboot,sig:15
[ 441.874736] caller:test_absys
[ 441.877725] caller:ash
Please enter your choice: Read init scrips : sysfixtime boot system sysctl log fstab ql_
=== add initd, file=/etc/rc.d/K20ql_powerkey_shutdown, param=shutdown
=== add initd, file=/etc/rc.d/K21wpad, param=shutdown
=== add initd, file=/etc/rc.d/K90boot, param=shutdown
=== add initd, file=/etc/rc.d/K90forwarder, param=shutdown

```

5. 模块在已更新系统上运行后，执行 `cat /etc/quectel-project-version` 查询固件版本号是否已经更新至目标版本。

```

root@OpenWrt:/data# cat /etc/quectel-project-version
Project Name: EC200A-CNTA
Project Rev : EC200ACNTAR02A06M2GV01
Build Date: Dec 01 2022 14:46:11

```

6. 根据提示输入“1”查询当前升级状态为 NEEDSYNC；输入“4”查询当前运行系统为 B 系统；输入“3”同步当前未激活系统，将当前运行系统（B 系统）的固件同步至非激活系统（A 系统）中，再次输入“1”查询当前升级状态为 SUCCEED，即完成升级。

```

Please enter your choice: 4

[DBG][test_ql_absys_get_cur_active_part_222][937633] Current active part is B

```

```

root@OpenWrt:/data# ./test_absys
Quectel OTA API test sample, version : v0.0.1

0      API : ql_abfota_start_update
1      API : ql_abfota_get_update_status
2      API : ql_absys_switch
3      API : ql_absys_sync
4      API : ql_absys_get_cur_active_part
5      API : ql_absys_getstatus
-1     exit
Please enter your choice: 1

[DBG][test_get_fota_upgrade_info_172][712] Current fota progress: 100
[DBG][test_get_fota_upgrade_info_173][712] Current fota state: NEEDSYNC
[DBG][test_get_fota_upgrade_info_174][712] Current fota exit code: 0
Please enter your choice:

```

```

Please enter your choice:

0      API : ql_abfota_start_update
1      API : ql_abfota_get_update_status
2      API : ql_absys_switch
3      API : ql_absys_sync
4      API : ql_absys_get_cur_active_part
5      API : ql_absys_getstatus
-1     exit
Please enter your choice: 3

[DBG][test_ql_absys_sync_206][937620] do AB sync succeed
Please enter your choice: 1

[DBG][test_get_fota_upgrade_info_172][937625] Current fota progress: 100
[DBG][test_get_fota_upgrade_info_173][937625] Current fota state: SUCCEED
[DBG][test_get_fota_upgrade_info_174][937625] Current fota exit code: 0

```

3.6.5. AB 系统非激活分区损坏查询及恢复处理

当系统使用过程中发生分区固件损坏，以 Kernel-b 分区损坏为例，则当前激活的 B 系统在下次重启后校验 Kernel-b 分区固件为损坏，并将损坏分区固件标记写入 flash flag 分区中记录，并触发系统切换到 A 系统启动，此时用户需输入“4”查看当前激活分区发生改变，再输入“5”查询非激活 B 系统的分区损坏位置，最后输入“3”进行分区同步恢复。

示例如下：

1. 当前运行的激活分区是 B 系统，A/B 系统分区状态为 succeed。

```

root@OpenWrt:/overlay# ./test_absys
Quectel OTA API test sample, version : v0.0.1

0      API : ql_abfota_start_update
1      API : ql_abfota_get_update_status
2      API : ql_absys_switch
3      API : ql_absys_sync
4      API : ql_absys_get_cur_active_part
5      API : ql_absys_getstatus
-1     exit
Please enter your choice: 1

[DBG][test_get_fota_upgrade_info_172][950869] Current fota progress: 100
[DBG][test_get_fota_upgrade_info_173][950869] Current fota state: SUCCEED
[DBG][test_get_fota_upgrade_info_174][950869] Current fota exit code: 0
Please enter your choice: 4

[DBG][test_ql_absys_get_cur_active_part_222][950872] Current active part is B
Please enter your choice: 5

[DBG][test_ql_absys_getstatus_122][950899] absys partition status : succeed

```

2. 破坏 Kernel-b 分区固件，重新启动。

```

mtd13: 00040000 00020000 "dtim-b"
mtd14: 00f00000 00020000 "cpimage-b"
mtd15: 000c0000 00020000 "u-boot-b"
mtd16: 00500000 00020000 "kernel-b"
mtd17: 01400000 00020000 "rootfs-b"
mtd18: 01400000 00020000 "oem_data-b"
mtd19: 00100000 00020000 "asr_flag"
mtd20: 00040000 00020000 "device_info"
mtd21: 00040000 00020000 "cust_info"
mtd22: 07660000 00020000 "rootfs_data"
mtd23: 00d20000 00020000 "QUEC_BBM"
root@OpenWrt:/overlay#
root@OpenWrt:/overlay# mtd erase /dev/mtd16
Unlocking /dev/mtd16 ...
Erasing /dev/mtd16 ...
root@OpenWrt:/overlay# hexdump -C /dev/mtd16
00000000  ff ff ff ff ff ff ff ff  ff ff ff ff ff ff ff ff  |.....|

```

3. 重启后，输入“4”查询可知当前激活系统为 A 系统，说明 Kernel-b 的破坏导致 AB 系统校验 kernel-b 失败，从而发生了系统切换。输入“5”，查询 A/B 系统的状态为 *damaged*，发生分区损坏的位置在 Kernel-b，则需输入“3”进行同步。同步完成后再输入“5”进行系统状态检查，发现恢复为 *succeed* 状态，此时 Kernel-b 被恢复。

```

Please enter your choice:

0      API : ql_abfota_start_update
1      API : ql_abfota_get_update_status
2      API : ql_absys_switch
3      API : ql_absys_sync
4      API : ql_absys_get_cur_active_part
5      API : ql_absys_getstatus
-1     exit
Please enter your choice: 4

[DBG][test_ql_absys_get_cur_active_part_222][951654] Current active part is A
Please enter your choice: 5

[DBG][test_ql_absys_getstatus_126][951658] absys partition status : damaged
[DBG][test_ql_absys_getstatus_127][951658] absys partition damaged position : kernel-b
[DBG][test_ql_absys_getstatus_128][951658] absys needsync!!!

```

```

0      API : ql_abfota_start_update
1      API : ql_abfota_get_update_status
2      API : ql_absys_switch
3      API : ql_absys_sync
4      API : ql_absys_get_cur_active_part
5      API : ql_absys_getstatus
-1     exit
Please enter your choice: [ 63.052093] Timestamp is: 01-12 08:33:50.515502929 UTC
3

[DBG][test_ql_absys_sync_206][952434] do AB sync succeed
Please enter your choice: [ 70.471679] KERNEL-TEXT-CRC: 0x4891

0      API : ql_abfota_start_update
1      API : ql_abfota_get_update_status
2      API : ql_absys_switch
3      API : ql_absys_sync
4      API : ql_absys_get_cur_active_part
5      API : ql_absys_getstatus
-1     exit
Please enter your choice: 5

[DBG][test_ql_absys_getstatus_122][952440] absys partition status : succeed

```

3.7. 资源消耗评估

3.7.1. 内存消耗评估

A/B 系统升级采用全量升级方式，为流式升级，以数据包的形式进行发送、接收和写入，消耗内存很少。

升级前内存使用情况如下：

```

# cat /proc/meminfo
cat /proc/meminfo
MemTotal:      98052 kB
MemFree:       42860 kB
MemAvailable:  76728 kB
Buffers:       3960 kB
Cached:        33764 kB
SwapCached:    0 kB

```

升级中内存使用情况如下：

```

# cat /proc/meminfo
cat /proc/meminfo
MemTotal:      98052 kB
MemFree:       41256 kB
MemAvailable:  75932 kB
Buffers:       4112 kB
Cached:        34404 kB
SwapCached:    0 kB

```

如上图对比，可用内存及空闲内存均只有略微使用，可知 A/B 系统升级对内存的依赖较小。

3.7.2. Flash 空间消耗评估

Flash 空间消耗主要表现为升级包占用的空间。

全量升级包中包含 CPIOImage（3 个固件）、zImage、root、oem_data、u-boot 和 tos（安全环境）6 个分区的 8 个镜像，升级包大小约为 22 MB（不包含用户镜像文件）。用户可将自己的程序及镜像导入到 oem_data 分区中，则占用 Flash 总大小约为 22 MB + oem_data 的大小。

3.7.3. CPU 资源消耗评估

系统运行正常后，进行 A/B 系统升级，升级过程中 CPU loading 最大值达到 8 %，总体 CPU 占用较小；由于 A/B 系统升级的全过程由相关接口控制，完全将整个升级过程的执行权限交给用户，一般不会导致资源竞争问题。

在升级的 UPDATE 及 BACKUP 阶段进行 CPU 占用测试，CPU 使用率没有明显增加，load average 值略微上升，说明没有因为升级进程而过度影响 CPU 占用率，只是在任务等待上有所增加，可知 A/B 系统升级过程对 CPU 资源消耗影响较小。

4 附录 参考文档及术语缩写

表 4: 参考文档

文档名称
[1] Quectel_EC200A-CN(TA)_QuecOpen_快速开发指导

表 5: 术语缩写

术语	英文描述	中文描述
API	Application Programming Interface	应用程序编程接口
CPU	Central Processing Unit	中央处理器
eMMC	Embedded Multimedia Card	嵌入式多媒体卡
FOTA	Firmware Over-The-Air	固件空中升级
IoT	Internet of Things	物联网
MTD	Memory Technology Device	存储器技术设备
OTA	Over-the-air	空中升级
SDK	Software Development Kit	软件开发工具包